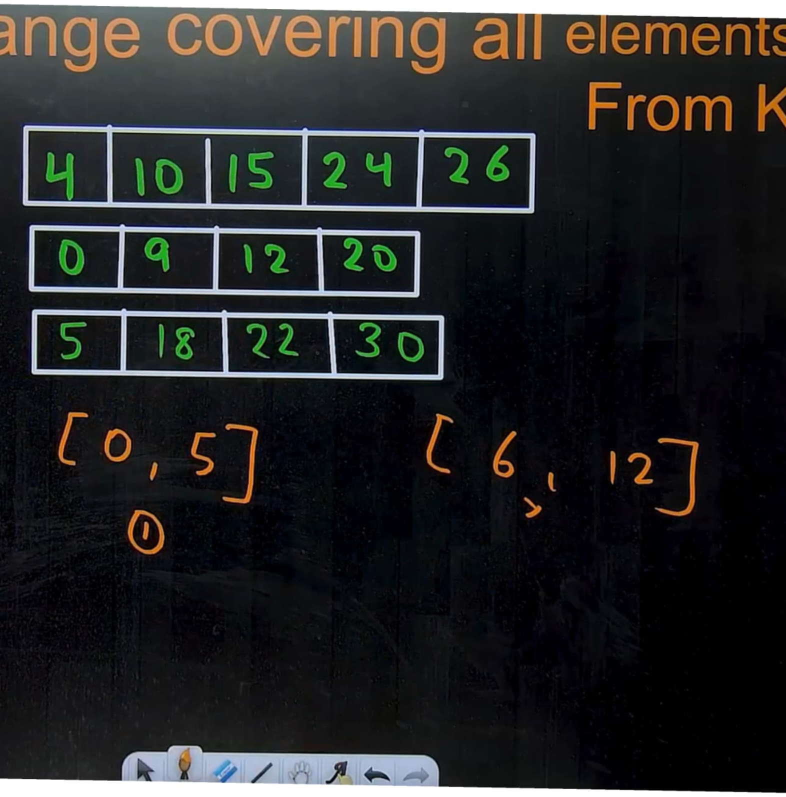
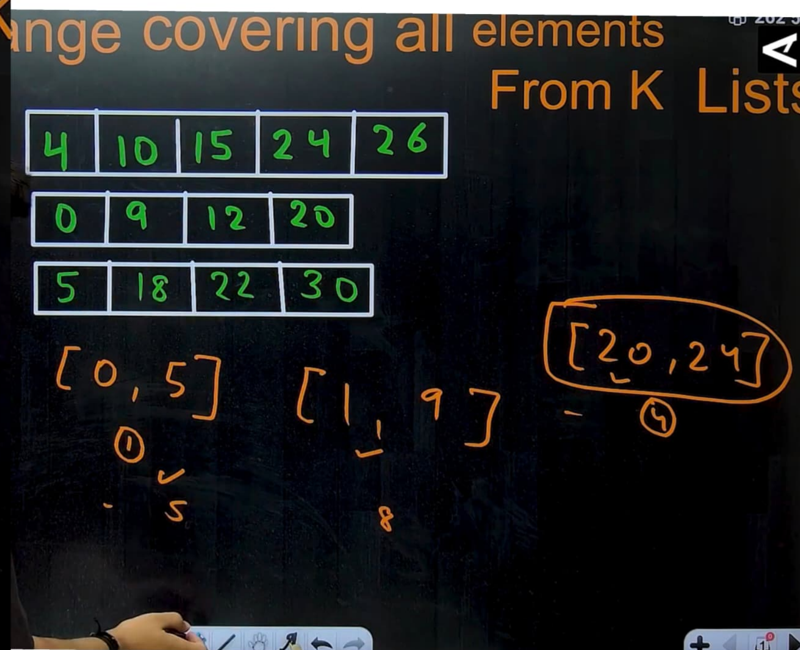
Smallest Range covering all elements From K Lists

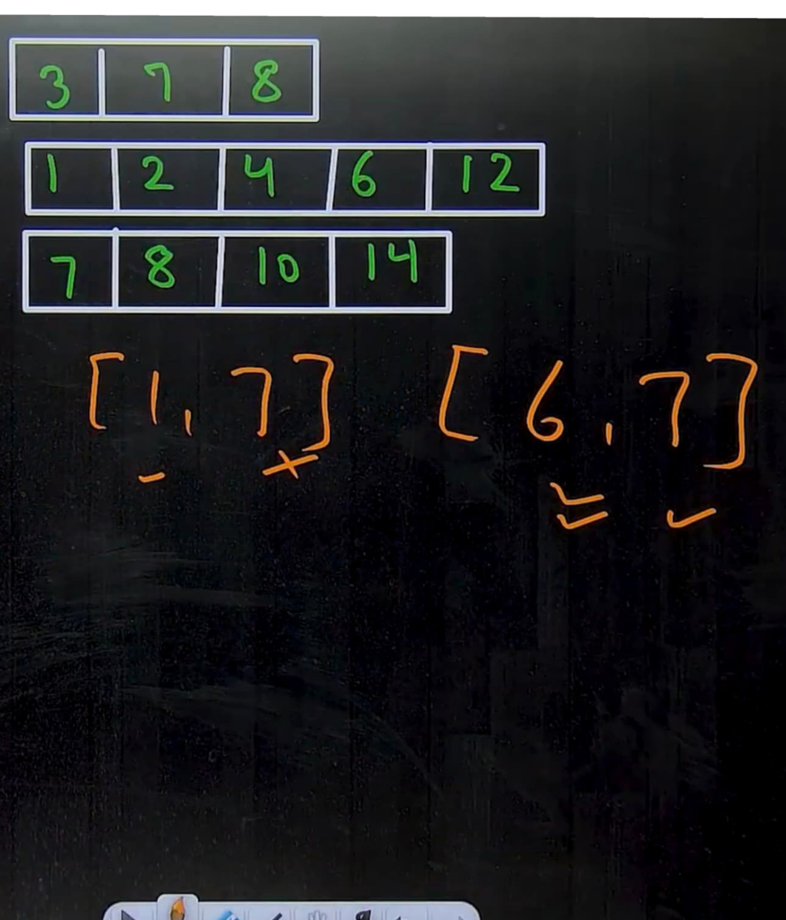
4	10	15	24	26
0	9	12	20	
5	18	22	30	

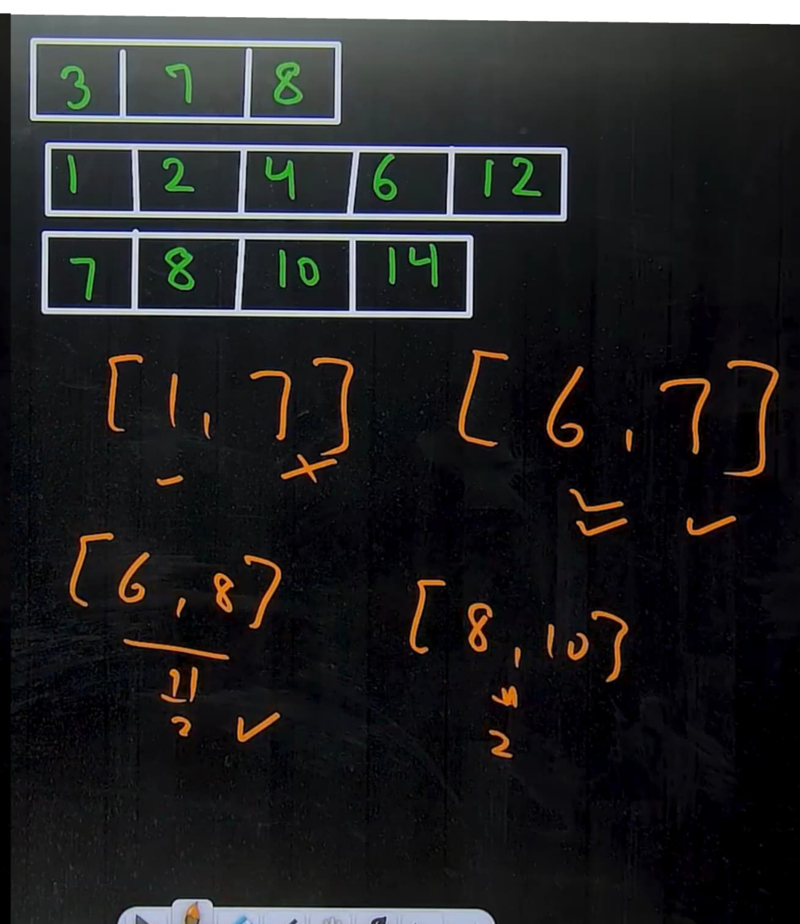
[0, 5]
1

[2, 6]
×


ange covering all elements
From K
4 10 15 24 26
0 9 12 20
5 18 22 30
[0, 5]
1
[6, 12]


nge covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
[0, 5]
1
5
[1, 9]
8
[20, 24]
4


3 7 8
1 2 4 6 12
7 8 10 14
[1, 7]
[6, 7]


3 7 8
1 2 4 6 12
7 8 10 14
[1, 7]
[6, 7]
[6, 8]
2
[8, 10]
2

Smallest Range covering all elements From K Lists

4	10	15	24	26

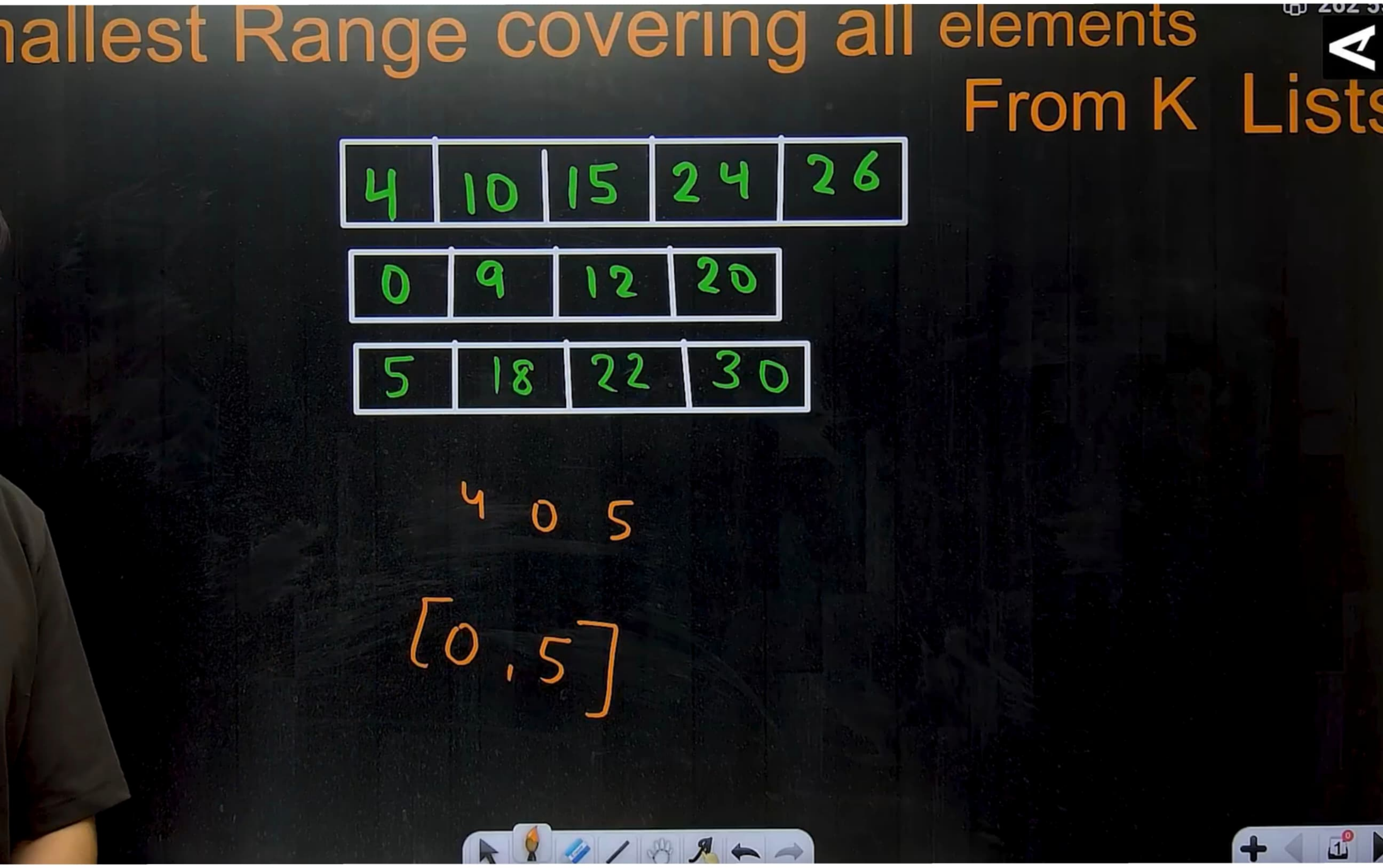
0	9	12	20

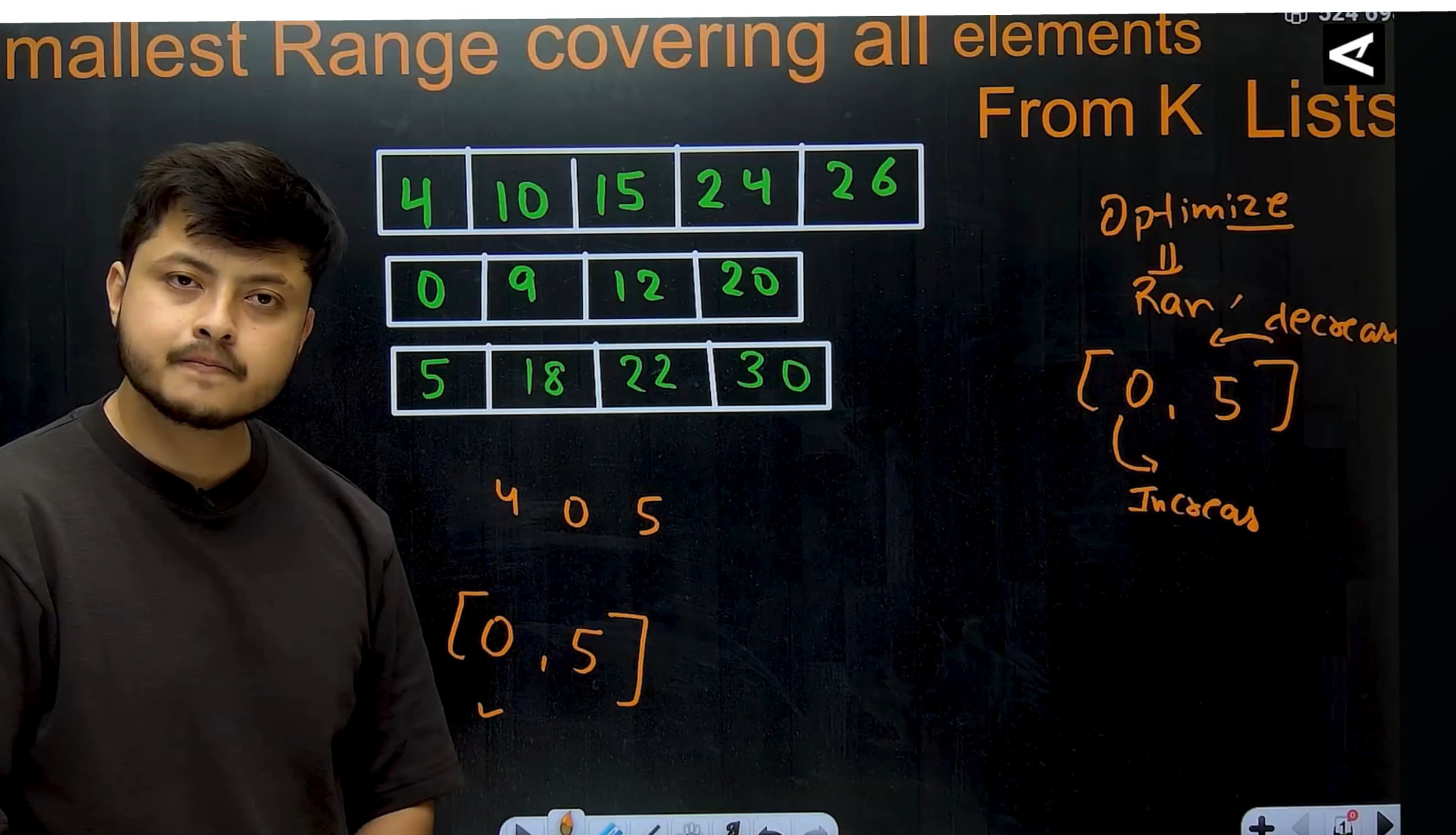
5	18	22	30

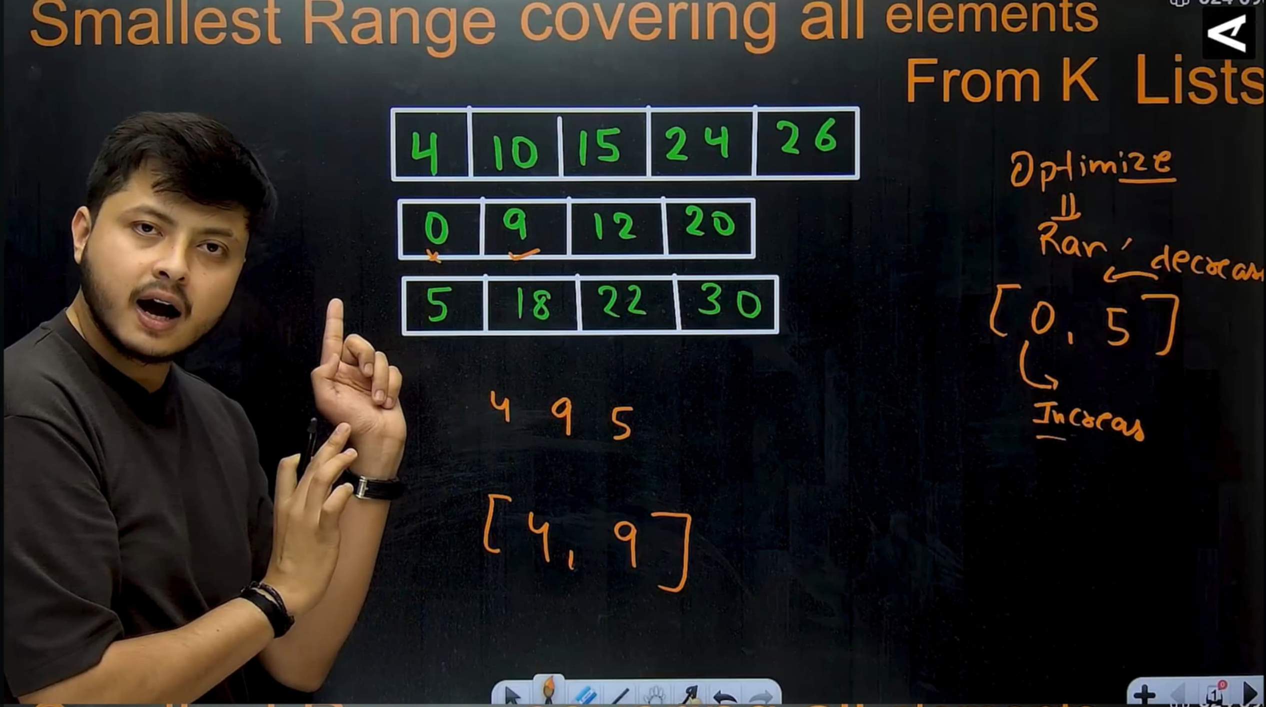
4	10	15	24	26	0	9	12	20	5	18	22	30

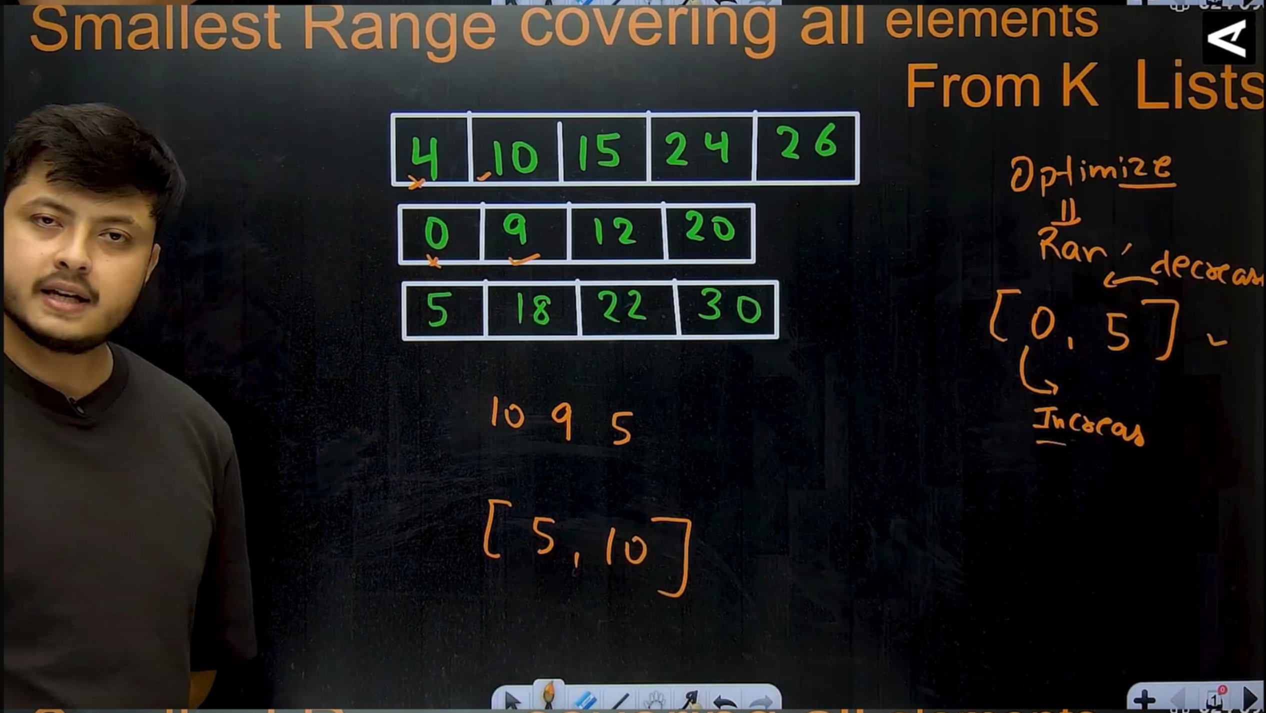
$K \times n$ [4, 10] [4, 15]

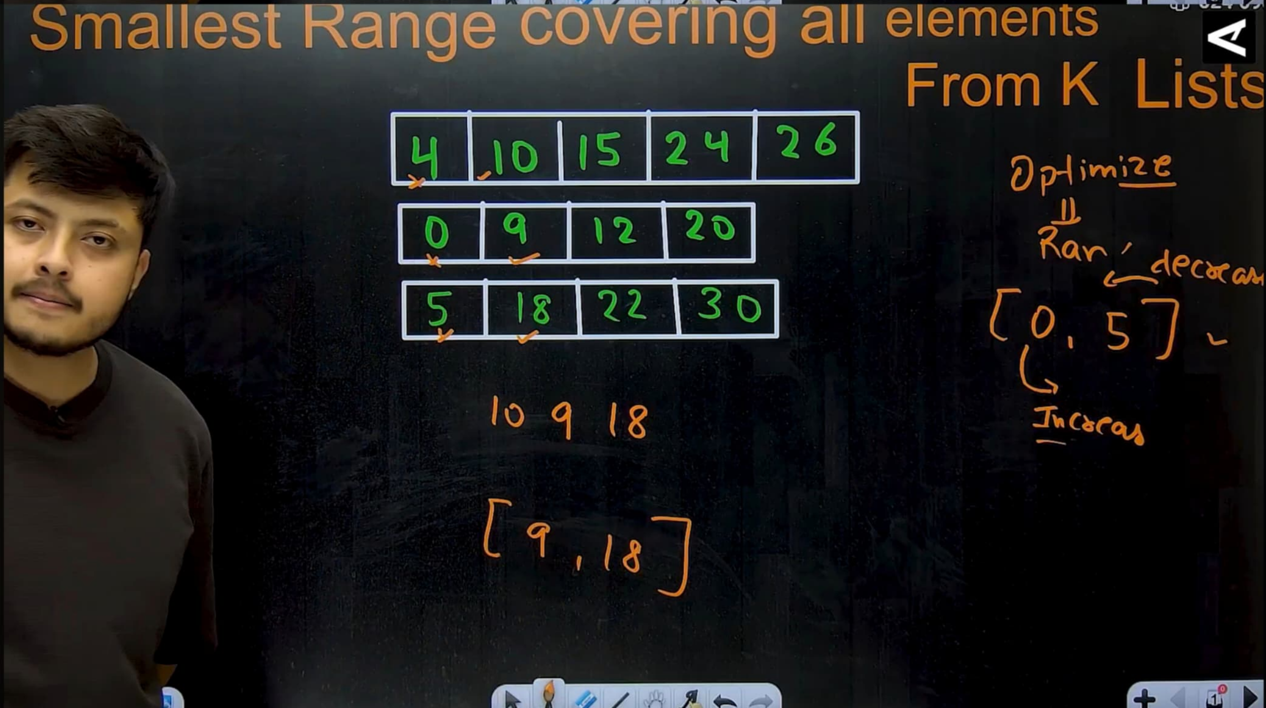
$(nK)^2, nK \rightarrow O(nK)^3$

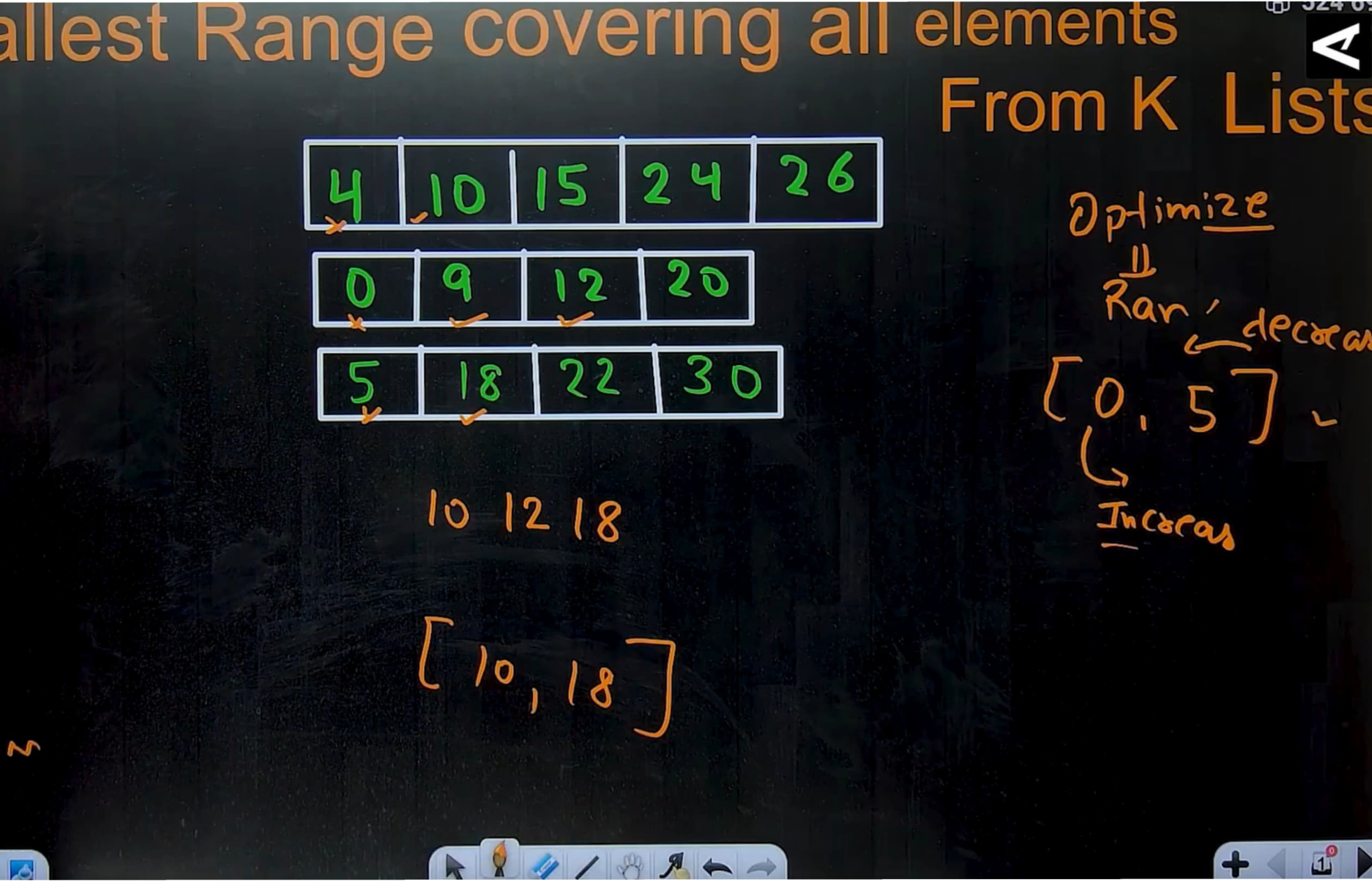

mallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
4 0 5
[0, 5]

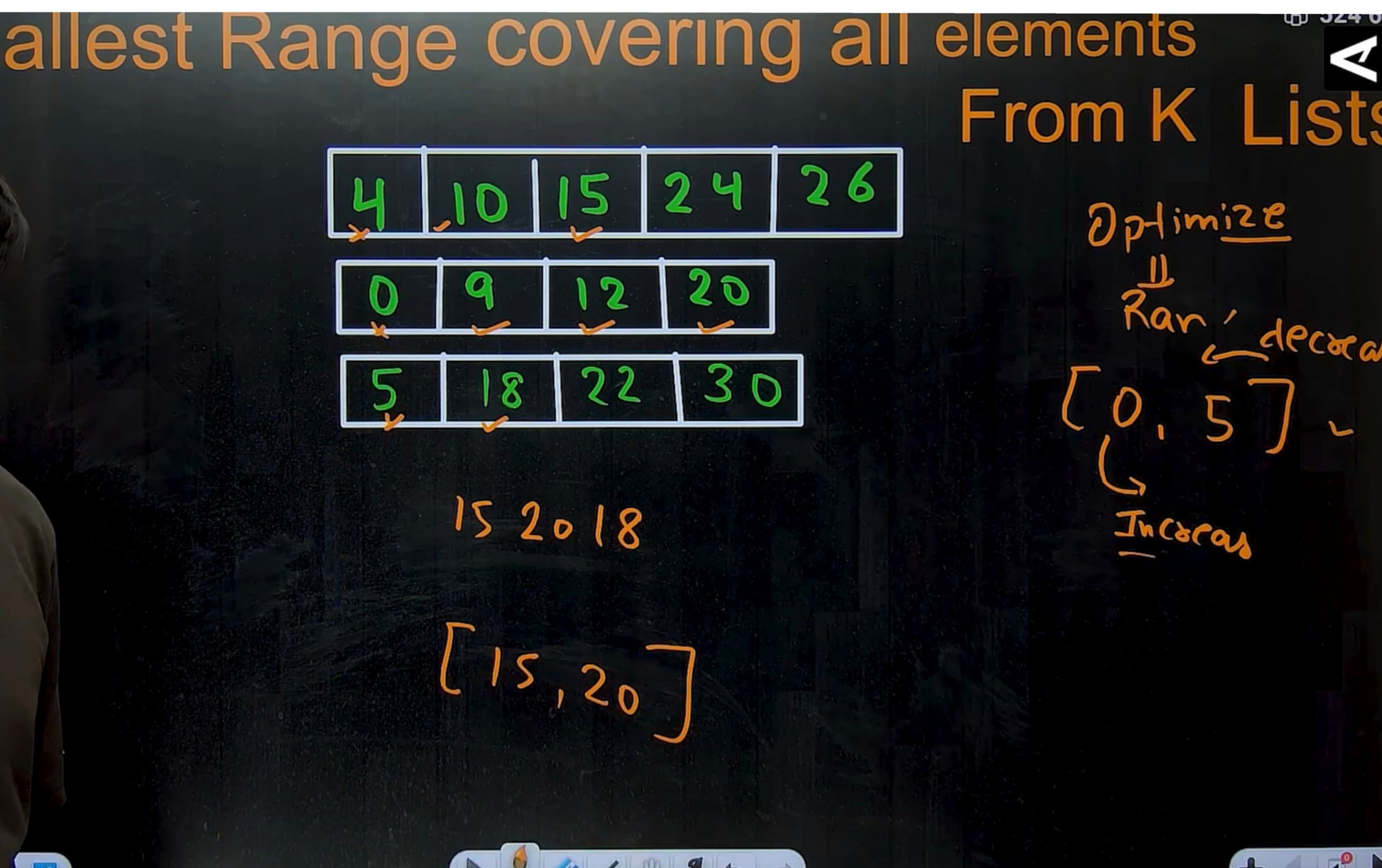

mallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
4 0 5
[0, 5]
Optimize
Ran
decreas
[0, 5]
Increas

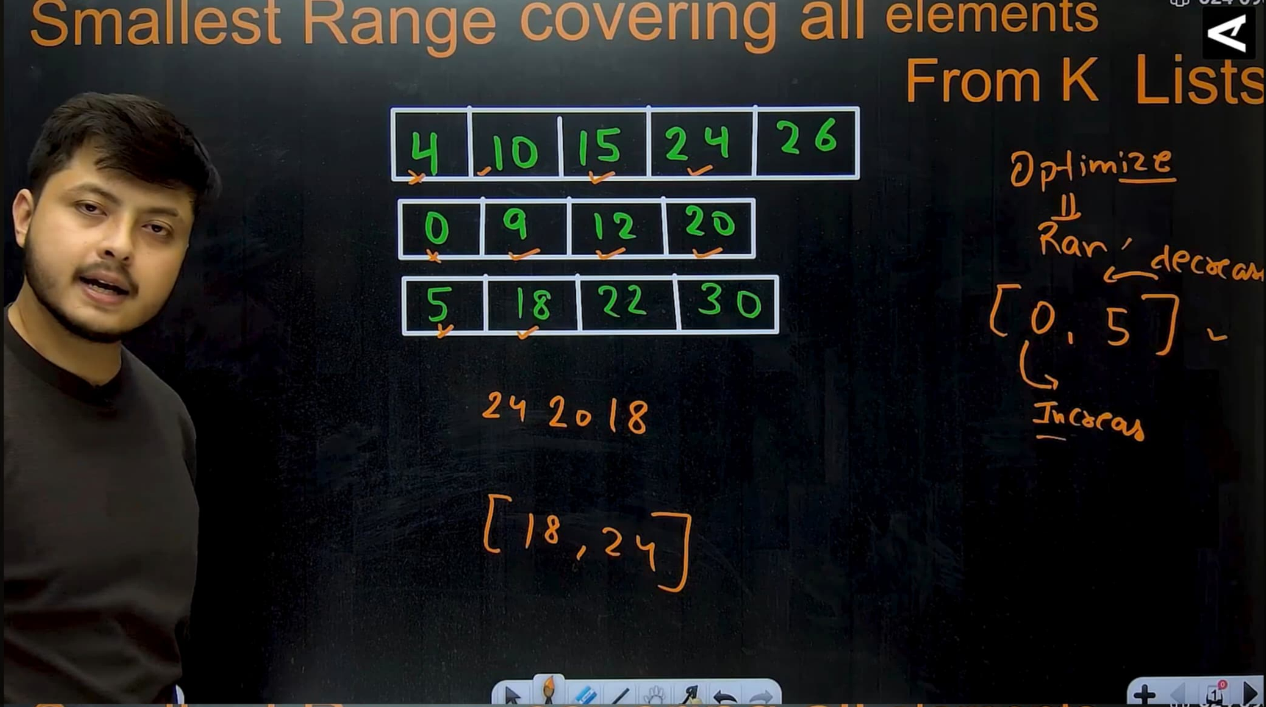

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
4 9 5
[4, 9]
Optimize
Ran
[0, 5]
Increas

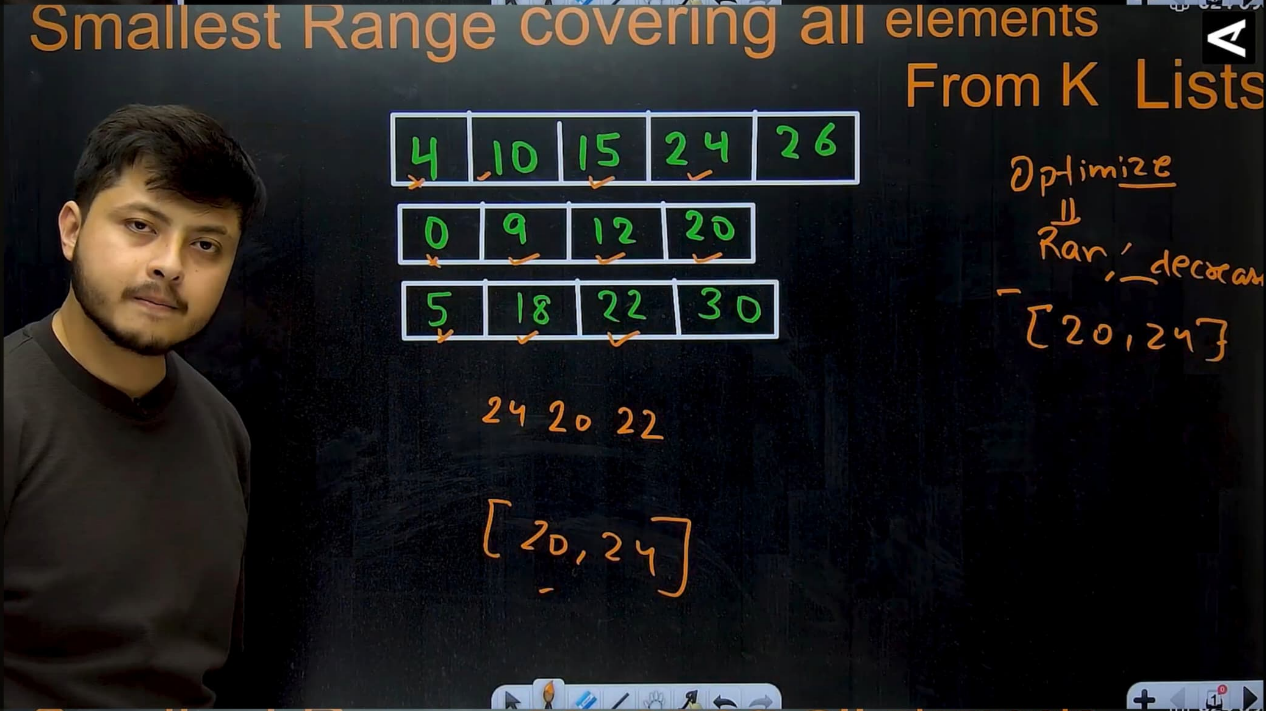

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
10 9 5
[5, 10]
Optimize
Ran
[0, 5]
Increas

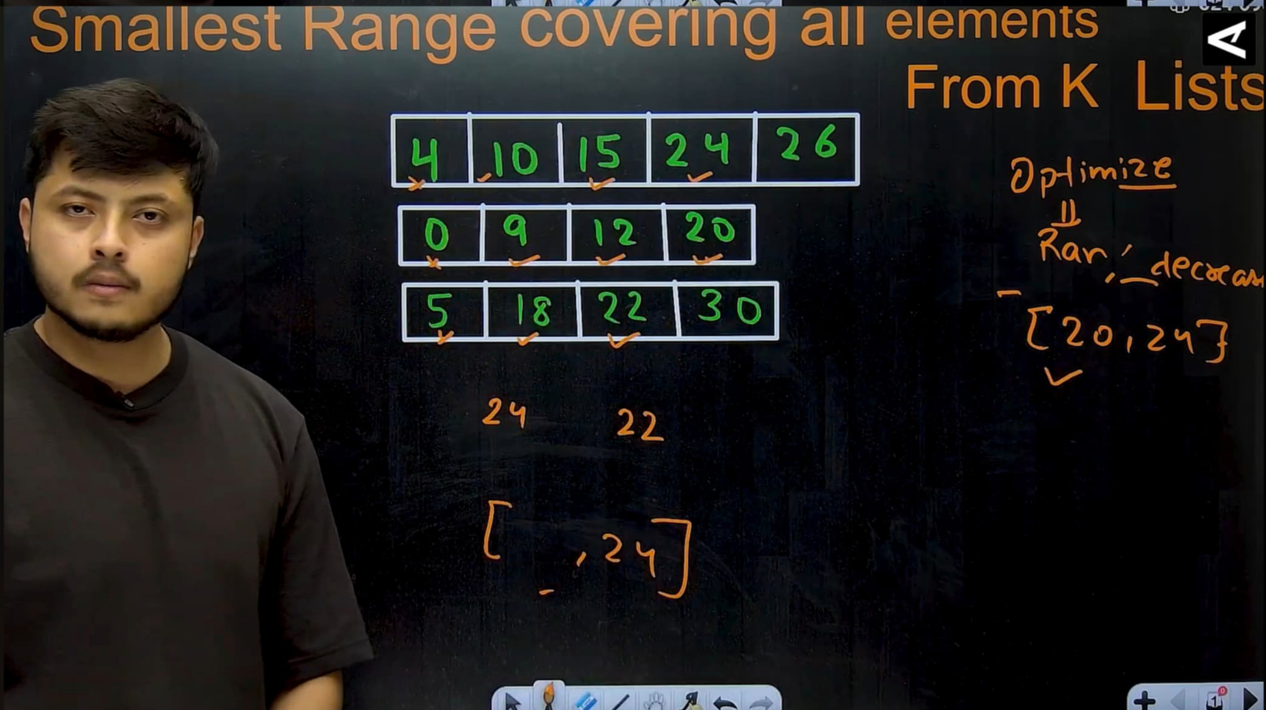

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
10 9 18
[9, 18]
Optimize
Ran
[0, 5]
Increas

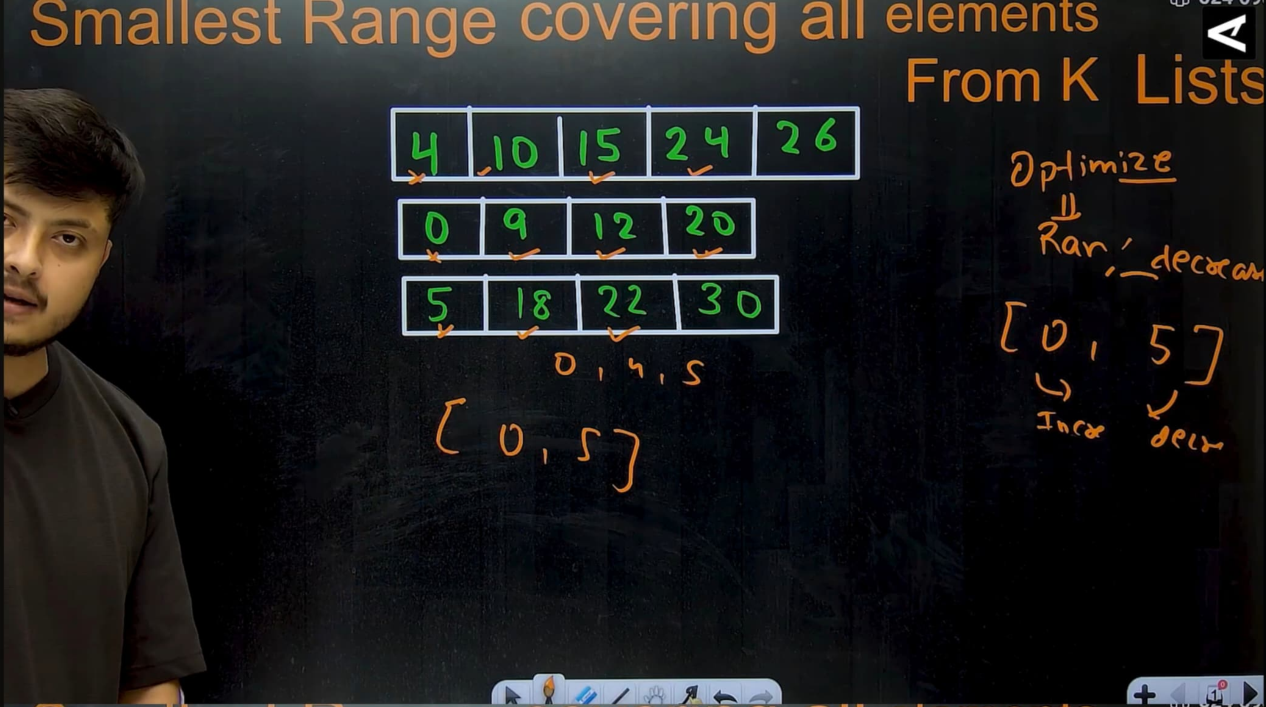

allest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
10 12 18
[10, 18]
Optimize
Ran
decreas
[0, 5]
Increas

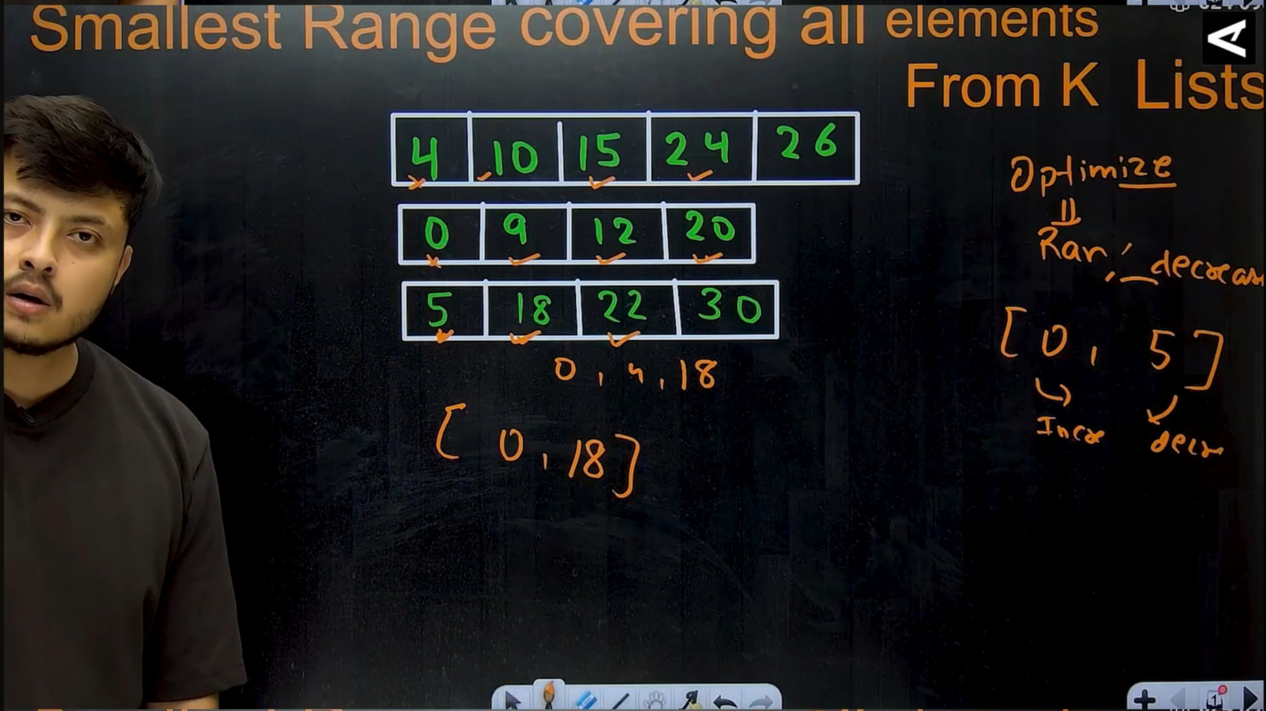

allest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
15 20 18
[15, 20]
Optimize
Ran
decreas
[0, 5]
Increas

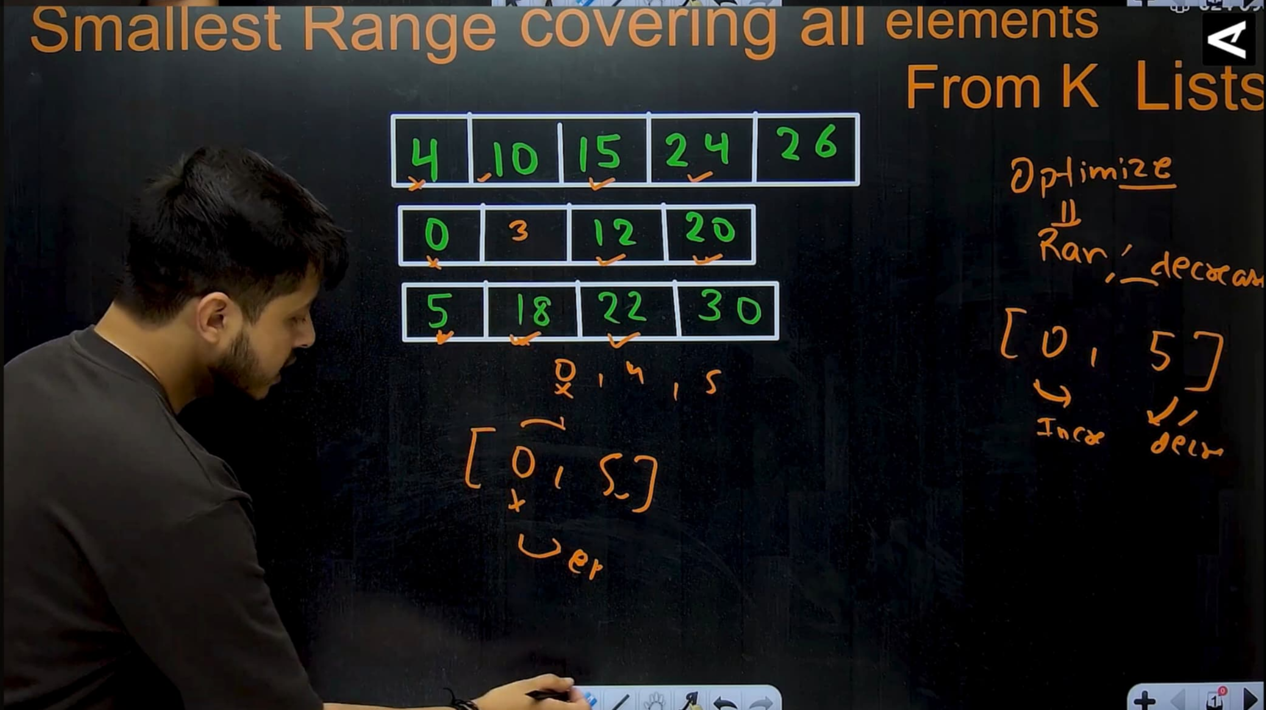

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
Optimize
Ran decreas
[0, 5]
Increas
24 20 18
[18, 24]

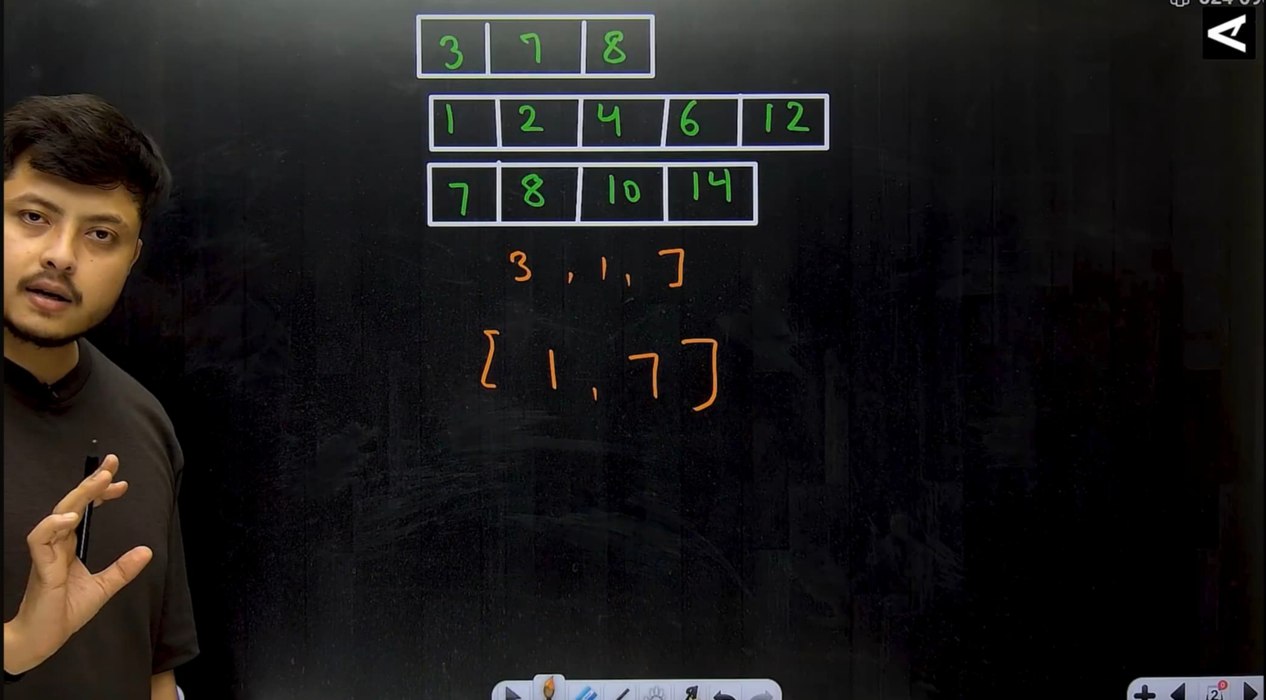

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
Optimize
Ran decreas
[20, 24]
24 20 22
[20, 24]

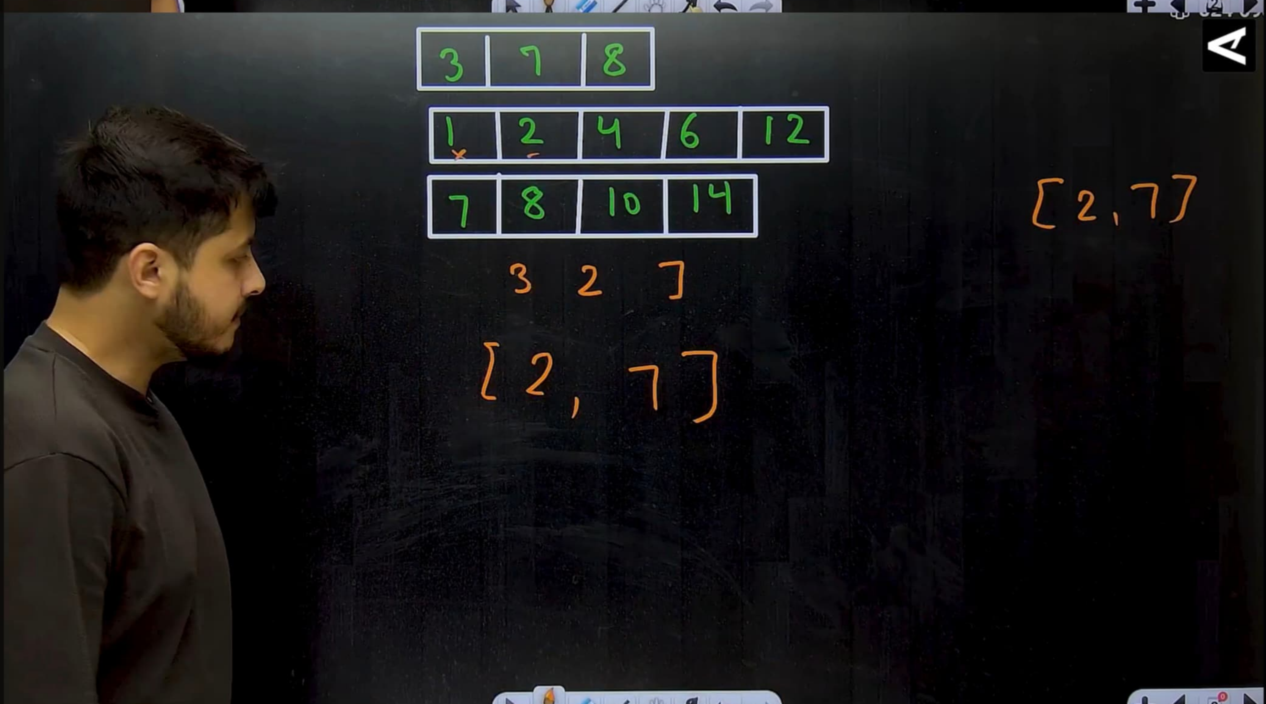

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
Optimize
Ran decreas
[20, 24]
24 22
[, 24]

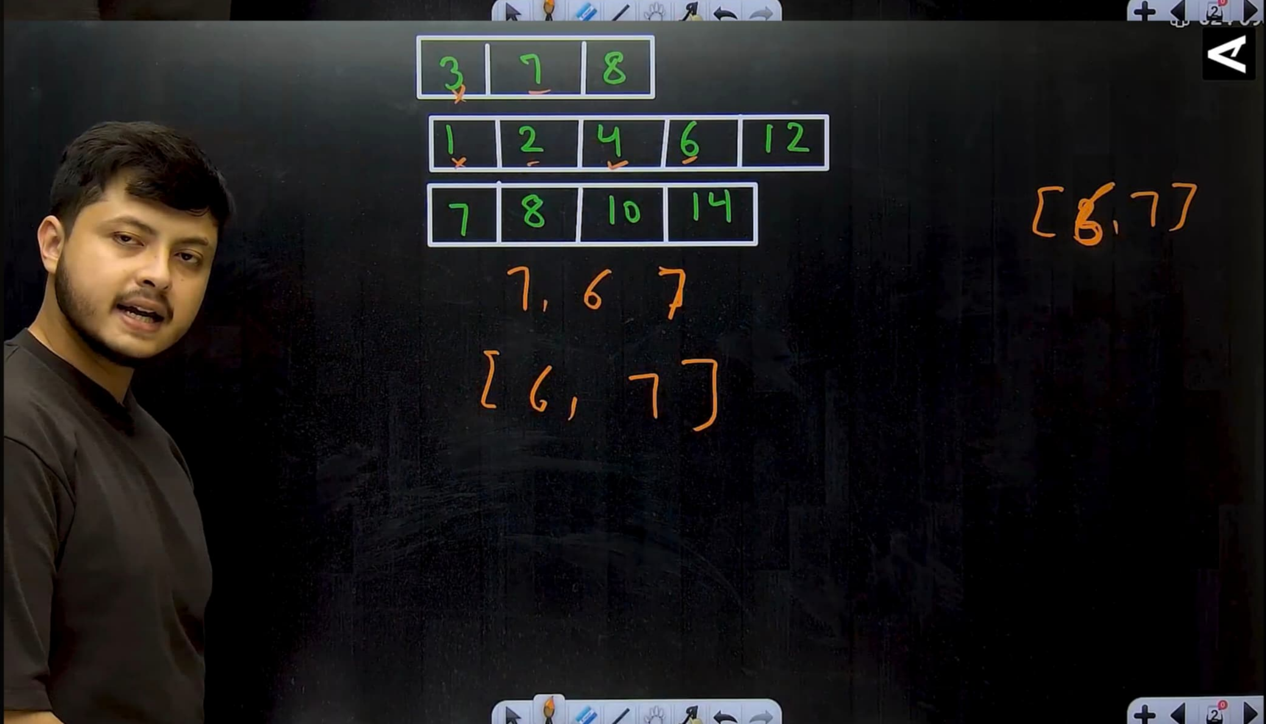

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
0, 4, 5
[0, 5]
Optimize
Ran, decrease
[0, 5]
Incr decr

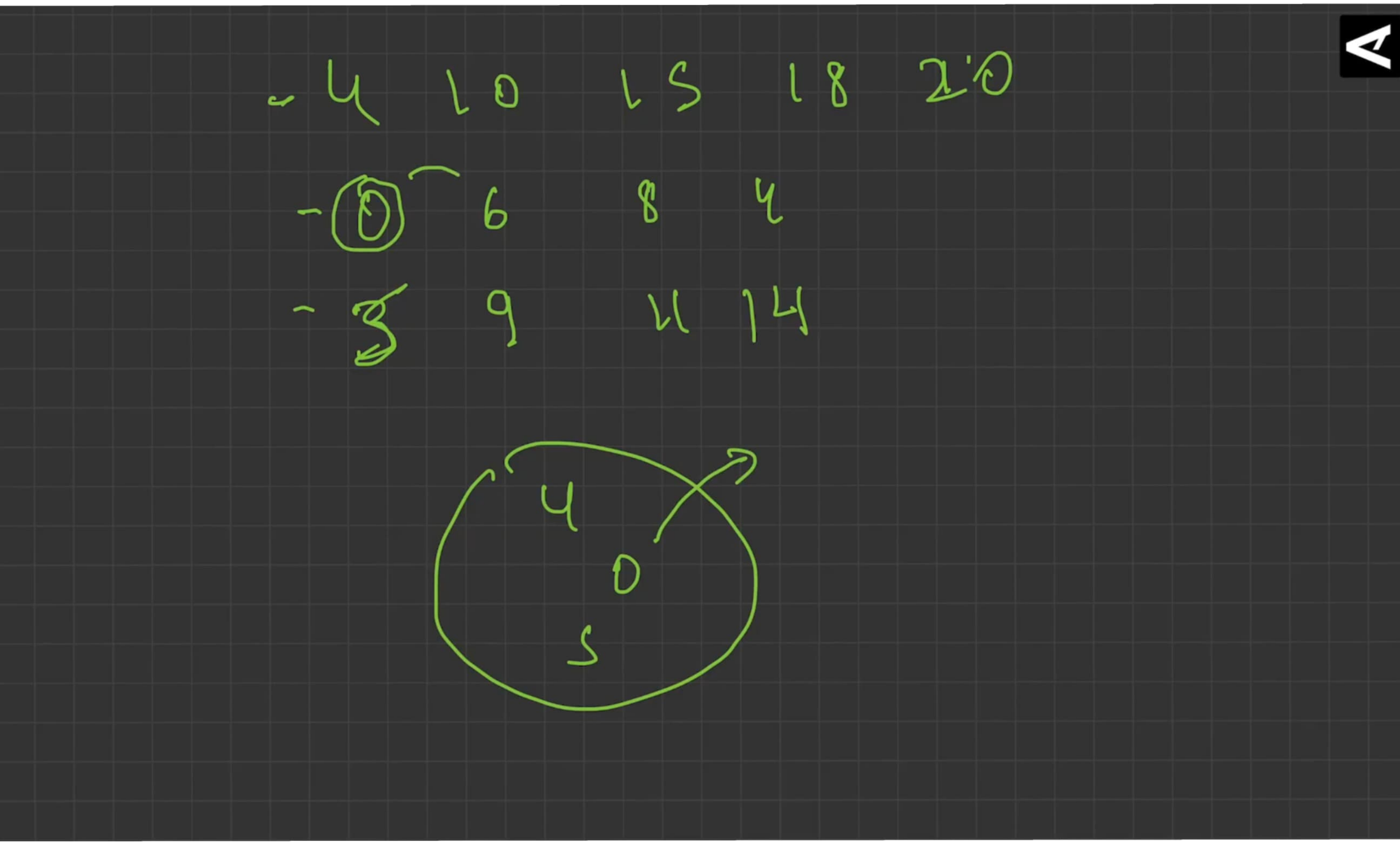

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 9 12 20
5 18 22 30
0, 4, 18
[0, 18]
Optimize
Ran, decrease
[0, 5]
Incr decr

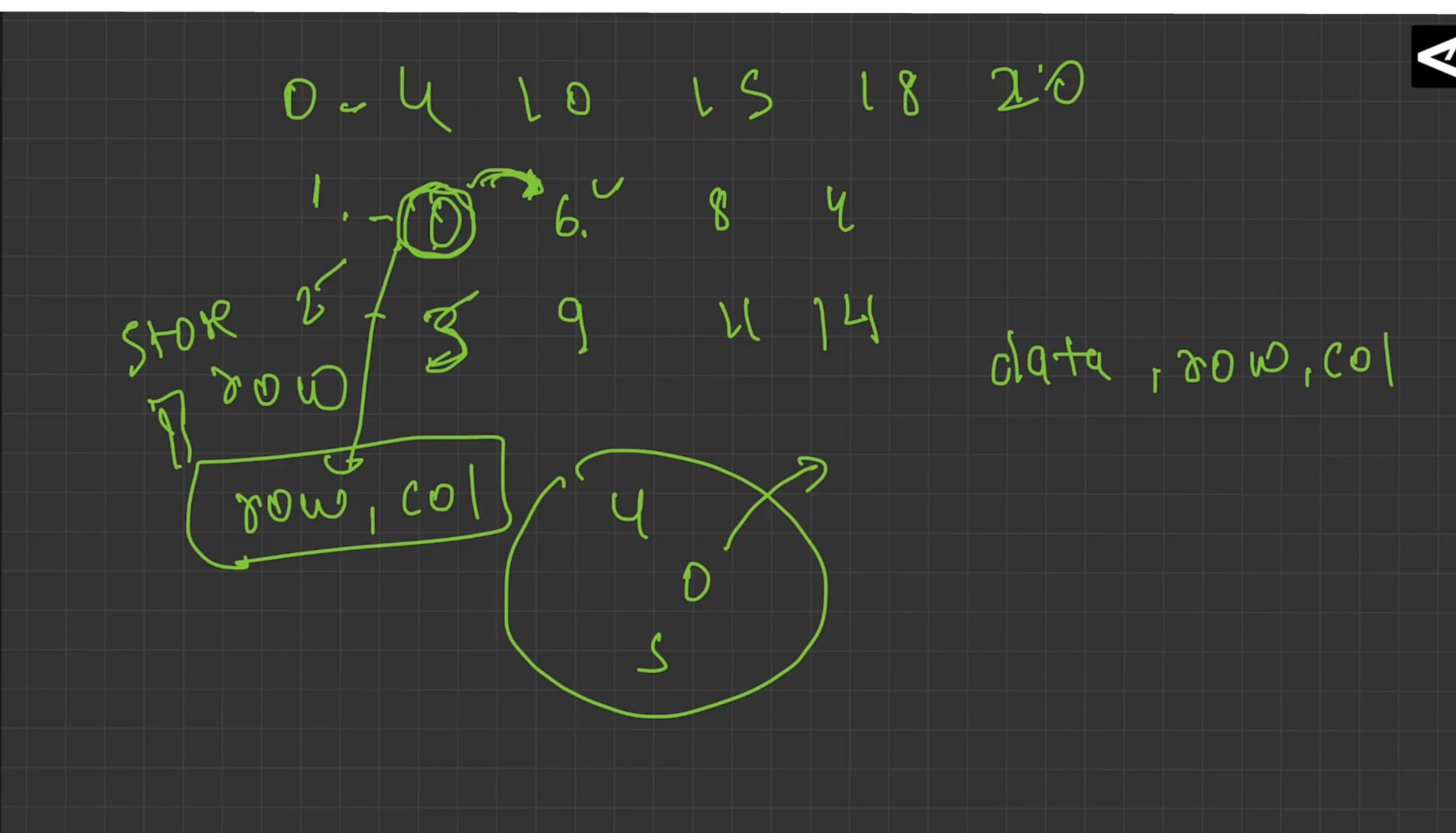

Smallest Range covering all elements
From K Lists
4 10 15 24 26
0 3 12 20
5 18 22 30
0, 4, 5
[0, 5]
er
Optimize
Ran, decrease
[0, 5]
Incr decr


3 7 8
1 2 4 6 12
7 8 10 14
3 , 1 , 7
[1 , 7]


3 7 8
1 2 4 6 12
7 8 10 14
3 2 7
[2, 7]
[2, 7]


3 7 8
1 2 4 6 12
7 8 10 14
7, 6 7
[6, 7]
[6, 7]


-4 10 15 18 20
-0 6 8 4
-3 9 11 14
4
0
5


0 -4 10 15 18 20
1. -0 6. 8 4
2 -3 9 11 14
STOR
row
[]
row, col
data, row, col
4
0
5

Problem List

Description

632. Smallest Range Covering Elements from K Lists

Solved

Hard | Topics | Companies

You have `k` lists of sorted integers in **non-decreasing order**. Find the **smallest** range that includes at least one number from each of the `k` lists.

We define the range `[a, b]` is smaller than range `[c, d]` if `b - a < d - c` **or** `a < c` if `b - a == d - c`.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],
[5,18,22,30]]
Output: [20,24]
Explanation:
List 1: [4, 10, 15, 24,26], 24 is in range
[20,24].
List 2: [0, 9, 12, 20], 20 is in range
[20,24]
```

3.4K | 13

Editorial | Solutions | Submissions

Run Submit 0

Code

C++ Auto

```cpp
class Solution {
public:
    vector<int> smallestRange(vector<vector<int>>& nums) {

        // min heap
        // value , row, col
        pair<int, pair<int,int>>
        priority_queue<int, vector<int>, greater<int>>p;

    }
};
```

Saving...

Ln 6,

Testcase | Test Result

You have `k` lists of
decreasing order.
at least one numbe

We define the rang
`d]` if `b - a < d -`

Example 1:

```
Input: nums =
[0,9,12,20],
Output: [20,2
Explanation:
List 1: [4, 1
range [20,24]
List 2: [0, 9
[20,24].
List 3: [5, 1
[20,24].
```

Example 2:

```
Input: nums =
Output: [1,1]
```

Constraints:

- `nums.length ==`

3.4K

Submission

```cpp
class Solution {
public:
    vector<int> smallestRange(vector<vector<int>>& nums) {

        // min heap
        // value , row, col

        priority_queue<pair<int, pair<int,int>>, vector<pair<int, pair<int,int>>>, gr

        int minimum , maximum = INT_MIN;
        // insert first element of each row into heap
        for(int i=0;i<nums.size();i++)
        {
            p.push(make_pair(nums[i][0], make_pair(i,0)));
            maximum = max(maximum, nums[i][0]);
        }

        minimum = p.top().first;
        vector<int>ans(2);
        ans[0] = minimum;
```

Saved

Ln 39,

Testcase | Test Result

You have k lists of
decreasing order.
at least one numbe

We define the rang
d] if b - a < d -

Example 1:

Input: nums =
[0,9,12,20],
Output: [20,2
Explanation:
List 1: [4, 1
range [20,24]
List 2: [0, 9
[20,24].
List 3: [5, 1
[20,24].

Example 2:

Input: nums =
Output: [1,1]

Constraints:

- nums.length ==

3.4K

```
  vector<int>ans(2);
  ans[0] = minimum;
  ans[1] = maximum;
 pair<int, pair<int,int>>temp;
 int row, col , elem;

  while(p.size()==nums.size())
  {
     temp = p.top();
     p.pop();

     elem = temp.first;
     row = temp.second.first;
     col = temp.second.second;

     if(col+1< nums[row].size())
     {
       col ++;
       p.push(make_pair(nums[row][col], make_pair(row,col)));
       maximum = max(maximum, nums[row][col]);
```

Saved

Ln 15,

Submission

Testcase | Test Result

You have `k` lists of
decreasing order.
at least one numbe

We define the rang
`d]` if `b - a < d -`

Example 1:

```
Input: nums =
[0,9,12,20],[
Output: [20,2
Explanation:
List 1: [4, 1
range [20,24]
List 2: [0, 9
[20,24].
List 3: [5, 1
[20,24].
```

Example 2:

```
Input: nums =
Output: [1,1]
```

Constraints:

- `nums.length ==`

👍 3.4K

Submission

```
    ans[1] = maximum;
   pair<int, pair<int,int>>temp;
   int row, col , elem;

    while(p.size()==nums.size()// Intital size of Priority queue should not decrease
    {
       temp = p.top();
       p.pop();

       elem = temp.first;
       row = temp.second.first;
       col = temp.second.second;

       if(col+1< nums[row].size())
       {
         col ++;
         p.push(make_pair(nums[row][col], make_pair(row,col)));
         maximum = max(maximum, nums[row][col]);
         minimum = p.top().first;
```

Saved

Ln 25,

Testcase | Test Result

You have k lists of
decreasing order.
at least one numbe

We define the rang
d] if b - a < d -

Example 1:

Input: nums =
[0,9,12,20],
Output: [20,2
Explanation:
List 1: [4, 1
range [20,24]
List 2: [0, 9
[20,24].
List 3: [5, 1
[20,24].

Example 2:

Input: nums =
Output: [1,1]

Constraints:

- nums.length ==

👍 3.4K

```
    elem = temp.first;
    row = temp.second.first;
    col = temp.second.second;

    if(col+1< nums[row].size())
    {
      col ++;
      p.push(make_pair(nums[row][col], make_pair(row,col)));
      maximum = max(maximum, nums[row][col]);
      minimum = p.top().first;

      // If I have got smallest range
      if(maximum-minimum < ans[1]-ans[0])
      {
          ans[0] = minimum;
          ans[1] = maximum;
      }


    }
```

Saved

Ln 25,

Submission

Testcase | Test Result

You have k lists of
decreasing order.
at least one numbe

We define the rang
d] if b - a < d -

Example 1:

> **Input:** nums =
> [0,9,12,20],
> **Output:** [20,2
> **Explanation:**
> List 1: [4, 1
> range [20,24]
> List 2: [0, 9
> [20,24].
> List 3: [5, 1
> [20,24].

Example 2:

> **Input:** nums =
> **Output:** [1,1]

Constraints:

- nums.length ==

3.4K

```
            minimum = p.top().first;

            // If I have got smallest range
            if(maximum-minimum < ans[1]-ans[0])
            {
                ans[0] = minimum;
                ans[1] = maximum;
            }


        }
    }

    return ans;

}

```

Saved

Ln 52,

Submission

Testcase

Test Result

Problem List Run Submit 0

Description

Elements from K Lists

Hard Topics Companies

You have `k` lists of sorted integers in **non-decreasing order**. Find the **smallest** range that includes at least one number from each of the `k` lists.

We define the range `[a, b]` is smaller than range `[c, d]` if `b - a < d - c` **or** `a < c` if `b - a == d - c`.

Example 1:

> **Input:** nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
> **Output:** [20,24]
> **Explanation:**
> List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
> List 2: [0, 9, 12, 20], 20 is in range [20,24].
> List 3: [5, 18, 22, 30], 22 is in range [20,24].

Example 2:

3.4K 13

Editorial Solutions Submissions

Code

C++ Auto

```cpp
            minimum = p.top().first;

            // If I have got smallest range
            if(maximum-minimum < ans[1]-ans[0])
            {
                ans[0] = minimum;
                ans[1] = maximum;
            }


        }
    }

    // n*klogK

    return ans;
```

Saved

Ln 54,

Testcase Test Result